

The Guided Filter Stack for SEIRS and MERS: a KLI Assessment

PhyloPOMP.jl development notes

7 September 2026

Disclosure. Prepared with assistance from Claude (Anthropic) for L^AT_EX typesetting, code cross-referencing, and exposition. All claims about the implementation are checked directly against the source files cited throughout (`src/guide.jl`, `src/examples/seir_guided.jl`, `src/examples/mers_guided.jl`, `src/examples/mers_funs.jl`, `src/coloring.jl`, `src/fsmarkov.jl`) and against `src/examples/mers_filter_su` from which this note borrows notation and, where marked, prose.

Contents

1	Introduction and scope	1
2	The guide construction	2
3	The SEIRS guided filter	4
3.1	Singular part	5
3.2	Regular part	5
4	The MERS guided filter	6
5	The four kernels, formally	7
5.1	Current (unfloored) kernels	7
5.2	The retracted ε -floor (historical)	8
6	Mathematical assessment in KLI terms	9
7	Verdict on the MERS kernel choice	12
8	Summary table	13

1 Introduction and scope

PhyloPOMP.jl filters phylodynamic data by treating an observed genealogy (typed tips, internal coalescent nodes, and a root) as the obscured/latent-coloring data of a structured Markov genealogy process (SMGP), in the sense of King, Lin & Ionides (KLI), *Exact Phylodynamic Likelihood via Structured Markov Genealogy Processes*, arXiv:2405.17032 [1]. A particle filter simulates the underlying population process forward in time and, whenever the simulated event structure must be

reconciled with the observed genealogy, proposes a *coloring* of the lineages consistent with population membership; KLI Theorem 5 (with Lemma 3) guarantees that *any* proposal π_u that is positive wherever the target’s reduced compatibility factor $\alpha_u \Phi_u$ is positive yields an unbiased likelihood estimator once each proposed outcome is reweighted by the boost $B_u = \Phi_u / \pi_u$. This is the single fact underwriting everything below: kernel choice is a variance question, not a correctness question, and the repository’s four proposal kernels are four different answers to “how should π be chosen” for the *same* target filter.

This repository’s own vocabulary — **naive**, **soft**, **guided**, and **hard** — names an escalating ladder of proposals built on a *filter guide*: a reverse-time, finite-state Markov process over the demes that can carry a lineage, run backward over the genealogy to produce, at every branch point, a belief about which deme each lineage occupies. Naive never consults the guide. Soft, guided, and hard all do, with three different mathematical relationships between the guide’s information and the resulting proposal law. This note has two purposes:

1. **Explain the stack** for the two worked models in this repository — the single-host SEIR(S) model (`seir_guided.jl`) and the two-host Camel/Human MERS-CoV model (`mers_guided.jl`) — at the level of the guide construction (§2), the SEIRS filter’s singular and regular parts (§3), the MERS filter’s singular and regular parts (§4), and the four kernels’ formal definitions (§5).
2. **Assess the kernel choice for MERS** (§6–§7) in KLI’s own framework: what each kernel is doing to the variance of the likelihood estimator, what approximation the guide itself introduces relative to the (intractable) locally optimal proposal, and whether **GuidedMERS** — the repository’s working MERS kernel — is in fact the right default. The short answer, defended in §7, is yes, with two qualifications that are about the *target model’s parameters*, not the proposal.

On borrowed material. Some notation and some formulas in §5 are reproduced, with credit, from `mers_filter_suite.tex` §9.1 (“Guided-family particle-filter kernels: soft, guided, and hard”), the companion derivation document for the MERS filter. As that document’s own v31 changelog explains (and as §5 below reflects), part of §9.1’s original text has since been *retracted*: this note is careful to mark retracted material as such rather than presenting it as current.

2 The guide construction

Both worked models place the filter guide over a small deme set: {Expos, Infec} for SEIRS (only E and I individuals can be the ancestor of a sampled lineage or a coalescent event; S and R never appear as a lineage’s deme), and {Camel, Human} for MERS. The guide itself is a continuous-time finite-state Markov process on this deme set, `FMarkovProc` (`src/fsmarkov.jl`), constructed by `fsmarkov(...)` from a user-specified stationary distribution π (one weight per deme) and a set of pairwise conductances; internally it stores the generator $Q = \Pi(\text{conductances})$ (rows summing to zero) together with a symmetrized eigendecomposition (L, R , eigenvalues Λ) so that the transition semigroup at elapsed time s is available in closed form,

$$P(s) = L \operatorname{diag}(e^{\Lambda s}) R,$$

and `forward_action(m,s,X)` applies $P(s)$ to a matrix of column-beliefs X (one column per lineage). This is the guide’s own dynamics; it does *not* depend on the population state (S, E, I, R) or (S_c, I_c, S_h, I_h) — a point that matters for the assessment in §6.

Vocabulary and two symbol collisions. This note uses KLI’s own symbols $\alpha_u, \Phi_u, \pi_u, B_u, \ell, \text{col}_d$ (jump mark $u \in \mathcal{U}$, hazard α_u , reduced factor Φ_u , proposal π_u , boost $B_u = \Phi_u/\pi_u$) exactly as in `mers_filter_suite.tex`; KLI’s own coloring operators are σ (recolor/swap a lineage), χ (chop a lineage out of the tracked set), and κ (fork one lineage into two colored children) — this repository’s `swap!`, `chop!`, and `fork!` implement exactly these three, and `plant!` is the corresponding operation for introducing a fresh root lineage. Where `filter_pomp` plumbing is described, “ $X(t)$,” “ Y_n ,” `rprocess`, `rinit`, `dmeasure/logdmeasure` are `pomp`’s own vocabulary (King, Nguyen & Ionides [2]). “Naive,” “soft,” “guided,” “hard,” the relative hazard r_b , and (in §5.2) the retracted floor’s $\varepsilon, \kappa_\varepsilon$ are this repository’s own coinages, appearing in none of KLI or `pomp`. Two collisions are unavoidable and worth flagging once, here, rather than re-disambiguating at every occurrence: (a) KLI’s chop operator χ is a different object from this same model’s destructive-sampling *rate parameter* χ (SEIRS) / χ_c, χ_h (MERS) — both are standard in their respective sources, and which is meant is always clear from context (an operator acts on a coloring; a rate multiplies a count); (b) KLI’s driver is written β_u , distinct from the MERS/SEIR transmission-rate parameters $\beta, \beta_{cc}, \beta_{hh}, \beta_{hc}, \beta_{ch}$ — this note, like `mers_filter_suite.tex`, never needs β_u directly and so avoids the collision by not introducing that symbol at all. A third collision, specific to the retracted floor mechanism, is flagged where it arises in §5.2.

The backward sweep. `Guide(g::Genealogy, m::FSMarkovProc, knowledge!)` (`src/guide.jl`) builds one `GuideNode` per genealogy node by walking the nodes in *reverse* order — from the most recent tip backward toward the root, i.e. running the finite-state chain backward in calendar time from the present into the past. A vector `probs[:,ells]` of per-lineage deme-beliefs is carried across the sweep. At each node n (interval $[t_n, t_{\text{end}})$, with t_{end} the time of the previously processed, more recent node):

1. **Right-endpoint snapshot.** `target = probs[:,ells]` records the current beliefs about every lineage present on the interval, as they stand at the interval’s right (more recent) endpoint, *before* they are propagated across the interval; `dtarget = m.right_trans * target` pre-rotates this snapshot into the guide process’s eigenbasis (the R factor above), so that later hazard evaluations (below) need only multiply by $\text{diag}(e^{\Lambda t})$ and left-multiply by $A := L/\pi$ (elementwise), rather than repeating the rotation at every query time.
2. **Forward-propagate to the left endpoint.** `probs[:,ells] = forward_action(m, tend - t_n, probs[:,ells])` advances every present lineage’s belief backward across the interval’s duration, landing at t_n (the interval’s left, i.e. older, endpoint). `present = probs[:,chillins]` then records this propagated belief restricted to n ’s *children* — exactly the “forward-propagated left-endpoint beliefs” used by the singular (branch-point) proposals in §3–§4.
3. **Resolve or fuse the parent.** `knowledge!(probs[:,parlin]; deme, type, time)` is asked whether the genealogy *pins* the parent lineage’s deme outright. For SEIRS (`seir_guided.jl`’s `knowledge!`), this is true exactly at a `Sample` or an internal `Node`: both event types certify that the ancestral lineage was in `Infec` at that instant, since only infectious individuals are ever sampled and only infectious individuals transmit. When pinned, `demekron!` overwrites

the belief with a Kronecker delta on that deme. For MERS (**GuidedMERS**’s own **knowledge!** in **mers_guided.jl**, and the equivalent **knowledge!** that **mers_funs.jl** shares with **SoftMERS** and **HardMERS**), only a typed **Sample** pins the deme (to whichever host the metadata records); an internal **Node** does *not* pin, because a MERS coalescence can equally be a within-host transmission or a cross-host spillover, so the parent’s host is generally uncertain even once both children’s beliefs are known. When **knowledge!** returns false, **assimil!** instead *fuses* the children’s (already left-endpoint propagated) beliefs:

$$\text{parprob}[i] \propto \pi(i) \prod_{c \in \text{children}} \frac{\text{chilprobs}[i, c]}{\pi(i)},$$

normalized over i . This is a product-of-experts (independent-evidence) pooling rule: each child’s belief is treated as an independent likelihood for the parent’s deme, expressed relative to the guide’s own stationary distribution π so that π is not multiply counted as a prior once per child. (When a node has no children under this scheme — **isempty(chilprobs)** — the parent belief simply defaults to π itself.)

4. The node’s four data fields — **type**, **tbeg**= t_n , **tend**, **parlin**, **chillins**, **alllins**=**ells**, **present**, **target**, **dtarget** — are stored in **GuideNode**. **tend** is then reset to t_n and the tracked-lineage set **lins** has the children removed and the parent added, continuing the sweep one node further into the past.

Relative hazards. Between nodes, **relhaz/relhaz!** evaluate, at a query time t inside a **GuideNode**’s interval, the guide’s *relative* hazard of recoloring a given lineage b from deme i to deme j , relative to the hazard that the *same* transition would have at stationarity:

$$r_b = \frac{p_j(b)}{p_i(b)}, \quad p(b) = A(\text{diag}(e^{\Lambda(t_{\text{end}}-t)}) \text{dtarget}[:, b]),$$

i.e. the guide’s belief about lineage b ’s eventual (right-endpoint) deme, pushed backward to time t through the eigenbasis and rescaled by $A = L/\pi$. $r_b \approx 1$ when the guide carries no information about b beyond stationarity, and by construction $\sum_b r_b = \ell$ exactly at stationarity (each of the ℓ tracked lineages contributing a hazard of 1 relative to itself). These r_b are exactly the quantities the guided-family kernels of §5 use to steer proposals; **choose_branch** is the function that turns a vector of r_b ’s (together with an “identity” weight for the untracked population) into an actual categorical draw, and it has two overloads used differently by the three guided-family kernels (§5).

3 The SEIRS guided filter

The model implemented in **seir_guided.jl** is SEIR with a waning term $\omega R \rightarrow S$, i.e. effectively SEIRS (the module’s own **event_rates!** sums transmission, progression, recovery, waning, and sampling; **waning!** contributes ωR moving one R back to S). GuidedSEIR shares its architecture directly with GuidedMERS: a **singular_part!** that fires exactly at genealogy nodes (root, sample, internal coalescence) and a **regular_part!** Gillespie loop that fires strictly between them.

Documentation bug. GuidedSEIR’s module docstring reads: “...using a filter guide and ‘soft’ proposals. That is, when population-process events occur, the guide can steer those events preferentially onto (or away from) particular branches, but the overall event rate remains equal

to that of the underlying population process.” This sentence is copied verbatim from `SoftSEIR`’s module docstring and is *stale* in `seir_guided.jl`: the code does not implement the naive-aggregate-preserving Soft law at all. `regular_transmission!` and `regular_progression!` call `choose_branch(t, guide, node, n, cols, i, j)` (the `guide.jl` overload that recomputes hazards and normalizes the identity weight *jointly* with every branch’s r_b in a single categorical draw), which is precisely the Guided law of §5, and is described that way, correctly, in `mers_filter_suite.tex`’s implementation map (“Guided (`mers_guided.jl`) mirrors `seir_guided.jl`’s architecture: ...the identity-vs-branch choice made in one joint draw via `choose_branch`”). GuidedSEIR is, mathematically, the SEIR analogue of GuidedMERS, not of SoftMERS; only its module docstring says otherwise.

3.1 Singular part

`singular_root!` draws the root lineage’s deme from `node.present[:, 1]  $\odot$  (n -  $\ell$ )`, where $n = (E, I)$ are the current population counts and $\ell = (\ell_E, \ell_I)$ the already-tracked counts: the guide’s belief is mixed with the *untracked capacity* of each deme, so a deme with zero free capacity ($E = \ell_E$ or $I = \ell_I$) can never be chosen for the root regardless of the guide’s opinion. If the draw succeeds with probability p , `ll` = $-\log p$ and the lineage is `plant!`-ed into the chosen deme; if every weight is zero, the particle dies (`ll` = $-\infty$).

At a sample node, `terminal_sample!` (no children) draws between non-destructive (ψ) and destructive (χ) sampling via `rcateg([ $\psi, \chi$ ])`, contributing $\log(\psi(I - \ell_I))$ or $\log(\chi I)$ respectively (post-`chop!` counts) plus the $-\log p$ draw cost, and decrementing I on a destructive draw; `inline_sample!` (one child, a non-destructive sample mid-lineage) contributes $\log \psi$ and re-labels the surviving child into `Infec` via `chop!(cols, Infec, parlin, Infec, chillin)`. At a coalescent node (`singular_branch!`, a birth event with $\beta SI/N$), the two children’s demes are drawn from

$$\text{branch_options}(x) = [x_{1,1}x_{2,2}, x_{1,2}x_{2,1}],$$

the two products of children’s present-probabilities corresponding to the two ways of assigning (Expos, Infec) across the two children — this *is* the guided two-child labeling proposal for a fork. With draw probability p ,

$$\text{ll} = \log\left(\frac{\beta SI}{N}\right) - \log p - \log(E \cdot I),$$

(counts E, I taken *after* $S- = 1, E+ = 1$): the final $-\log(E \cdot I)$ term is the KLI reduced compatibility factor ϕ_u for a birth whose production vector is $r = (1, 1)$ with *both* slots landing on already-tracked (observed) lineages — i.e. the observed children are two specific individuals out of E total exposed and I total infectious, respectively, with no free/untracked slot left over. As noted in §2’s vocabulary remark, `chop!`, `fork!`, and `swap!` here implement KLI’s coloring operators χ , κ , and σ respectively (`plant!` introduces a fresh root lineage); the KLI chop operator χ appearing in this sentence is not the sampling-rate parameter χ used two paragraphs above.

3.2 Regular part

Between nodes, `regular_part!` runs an ordinary Gillespie loop over four population-rate events (transmission, progression, recovery, waning) plus a fifth, purely-decay contribution from sampling ($(\psi + \chi)I$, which is never itself realized as a regular jump — sampling only happens at data nodes — so its entire rate is folded into λ continuously). Transmission and progression are proposed *jointly*

with a possible lineage recoloring via `choose_branch`: e.g. `regular_transmission!` draws among “no recolor” (an untracked S individual is infected) and “recolor tracked branch b from Infec to Expos” (interpreting the observed transmission as, in reverse time, exactly the moment one of the currently-tracked infectious lineages was itself created), with

$$\text{ll} = -\log p + \begin{cases} \log(1 - \ell_E/E), & b = 0, \\ \log(1 - \ell_I/I) - \log E, & b \neq 0, \end{cases}$$

(post-update E, I). Recovery is proposed at the *reduced* rate $\gamma I(1 - \ell_I/I)$ — i.e. only an untracked infectious individual may be proposed to recover, since recovering a tracked (sampled/internal) lineage is not observed — with the corresponding leftover rate $\gamma \ell_I$ returned to the decay λ rather than ever driving a jump; waning (ωR) carries no coloring consequence and no boost. The running log-likelihood accumulates $-\lambda \Delta t$ continuously between jumps, exactly the KLI decay term $-\int \lambda dt$.

This is not a plug-and-play pump filter. `filter_pump` assembles a `pump` object (King, Nguyen & Ionides [2]) with the latent state process $X(t) = (S, E, I, R)$, observations Y_n implicit in the genealogy’s node times, and the usual `rinit`/`rprocess`/`logdmeasure` components, so a plain bootstrap `pfilter` call runs on it exactly as on any other `pump` model. But the guide that `rprocess` consults through `userdata` is built by a call to `guide(gen,m,...)` that needs the population’s full rate structure (which events pin which demes, via `knowledge!`) to construct `present/target`, not merely a black-box simulator for $X(t)$. The filter is therefore *not* plug-and-play in `pump`’s sense: swapping in a new SEIRS-like model requires re-deriving `knowledge!`, the singular coloring operators, and the guide’s own deme set, not just supplying a new `rprocess`.

4 The MERS guided filter

`mers_guided.jl` implements the two-host analogue; it is now self-contained (it no longer includes `mers_funs.jl`), mirroring `seir_guided.jl`’s architecture. The regular process (`event_rates!` in `mers_guided.jl`) carries ten hazards: within-camel and within-human transmission (`cc`, `hh`), the two cross-host spillover directions (camel→human, human→camel), the two removal events (gated on $I_d > \ell_d$, exactly as SEIRS’s recovery), and four demography events (S_c/S_h births and deaths). The decay collects

$$\lambda = \chi_c I_c + \chi_h I_h + \gamma_c \ell_C + \gamma_c (I_c - \ell_C) \mathbb{1}_{I_c \leq \ell_C} + \gamma_h \ell_H + \gamma_h (I_h - \ell_H) \mathbb{1}_{I_h \leq \ell_H},$$

matching the SEIRS pattern of “sampling always decays, removal above threshold drives, removal at threshold decays.”

Spillover as a guided branch choice. The two spillover directions are exactly the guided `choose_branch` draws of §2: a camel→human event (`choose_branch(t,guide,node,I.c,cols,Camel,Human)`) either recolors no tracked camel lineage ($b = 0$, contributing $\log(1 - \ell_H/I_h)$ post-update) or recolors a specific tracked camel lineage into Human ($b \neq 0$, contributing $\log(1 - \ell_C/I_c) - \log(I_h)$ via `swap!(cols,Camel,Human,b)`); the human→camel direction is the mirror image. This is the MERS instance of the same joint identity/branch categorical used by SEIRS transmission and progression.

Within-deme transmission: the pair-exclusion correction. Unlike SEIRS transmission (which recolors an S individual, a deme with no tracked lineages), a within-camel birth (cc) can coalesce *two already-tracked* camel lineages, so its boost is not a single-slot ratio but the pair-exclusion factor

$$\log\left(1 - \frac{\ell_C(\ell_C - 1)}{I_C(I_C - 1)}\right),$$

the probability that neither of the two individuals involved in this particular birth is one of the $\binom{\ell_C}{2}$ already-tracked pairs (the hh case is the mirror). This is exactly $\log B_{\text{TCC}}$ from `mers_filter_suite.tex`'s worked filter equations.

Singular part. `mers_guided.jl`'s own `singular_part!` handles the root (a guide-and-free-capacity-weighted choice of Camel vs. Human, exactly analogous to SEIRS's root draw, but over $(I_c - \ell_C, I_h - \ell_H)$), destructive sampling (`chop!`, i.e. KLI's χ operator, plus the rate-parameter contribution $\log(\chi_d I_d)$, $d \in \{c, h\}$ — the two χ 's here are the operator/parameter collision flagged in §2 — read from the genealogy's own `deme` metadata, since a `GuideNode` carries no deme field of its own), and coalescent forks. At a fork whose parent is currently colored Camel, the three feasible outcomes — both children Camel (rate $\beta_{cc} S_c I_c / N_c$, guide weight $g_1 = \text{present}[1, 1] \text{present}[1, 2]$), or a cross-host spillover with either orientation (rate $\beta_{hc} S_h I_c / N_c$, guide weights $g_2 = \text{present}[1, 1] \text{present}[2, 2]$, $g_3 = \text{present}[2, 1] \text{present}[1, 2]$) — are drawn from `rcateg([ratecc $g_1$ , ratehc $g_2$ , ratehc $g_3$ ])`, mirroring the human-parent case with the roles exchanged. As in SEIRS, the final log-likelihood increment for the realized outcome is $\log(\text{rate}) - \log(I_c(I_c - 1)/2)$ (same-deme) or $\log(\text{rate}) - \log(I_c I_h)$ (cross-deme), the KLI reduced compatibility factors for, respectively, a same-deme pair and a cross-deme pair of already-tracked lineages.

5 The four kernels, formally

All four kernels — naive, soft, guided, hard — simulate the *same* target population process and are reweighted against the *same* target filter; only the proposal π_u (for soft/guided) or the auxiliary intensity κ_u (for hard) differs. Throughout, fix a source deme d with $I = I_d(y^-)$, $\ell = \ell_d(y^-)$ tracked lineages, and relative hazards $r_b \geq 0$, $b \in \text{col}_d(y^-)$, from §2.

Terminology. “Naive,” “soft,” “guided,” and “hard,” as before, are this repository's own names; $\pi_u, \Phi_u, B_u, \alpha_u, \ell_d, I_d, \text{col}_d$ are KLI's, used as in `mers_filter_suite.tex` §1/§4/§9.1.

5.1 Current (unfloored) kernels

As of the `mers_filter_suite.tex` v31 revision (17 August 2026), *every* MERS kernel — and, as confirmed directly against `seir_soft.jl/seir_guided.jl/seir_hard.jl`, every SEIR kernel, which never had a floor to begin with — uses *unfloored* base laws. There is no `proposal_floor` parameter, no `floored_shares/floored_branch_law/hard_branch_law` helper, and no local `floored_choose_branch` in `mers_guided.jl`; a direct grep of `src/examples/*.jl` and `src/*.jl` for `proposal_floor/epsilon` returns nothing. The three base laws are:

Soft. Preserves the naive aggregate identity/tracked split exactly,

$$\pi(0) = \frac{I - \ell}{I}, \quad \pi(b) = \frac{\ell}{I} \cdot \frac{r_b}{\sum_{b'} r_{b'}} \quad (b \in \text{col}_d(y^-)),$$

falling back to a uniform $1/\ell$ share within the tracked group when $\sum_{b'} r_{b'} = 0$. The guide changes *which* tracked branch is proposed; it never changes *how likely* the tracked group is as a whole. Implemented via `rcateg` on the raw `relhaz` outputs (`mers_soft.jl/seir_soft.jl`).

Guided. Normalizes the identity weight and every branch’s hazard *jointly*, in one categorical:

$$\pi(j) = \frac{w_j}{\sum_k w_k}, \quad (w_0, w_1, \dots, w_\ell) = (I - \ell, r_1, \dots, r_\ell).$$

This is exactly `choose_branch(t, guide, node, n, cols, i, j)` in `guide.jl` (§2): the identity weight $s_1 = n - \ell$ and the branch sum $s_2 = \sum_b r_b$ are combined into one total $s = s_1 + s_2$ before a single uniform draw on $[0, s]$ selects either the identity outcome (probability s_1/s) or a specific branch b (probability p_b/s). Because the identity weight competes directly against *every* r_b in one normalization, a single dominant guide hazard can push $\pi(0)$ far below the naive ratio $(I - \ell)/I$ — the guide reallocates mass *between* identity and tracked outcomes here, not only *within* the tracked group as in Soft. This is the mathematical distinction between Soft and Guided (stated the same way in `mers_filter_suite.tex`’s module docstrings for `SoftMERS` and `GuidedMERS`, quoted in §4’s introduction).

Hard. Uses the raw, unnormalized relative hazards as *auxiliary intensity* multipliers, so the auxiliary process need not carry the population process’s total rate. Following `mers_filter_suite.tex`, this note writes the auxiliary intensity as $\kappa(\cdot)$; this κ is *this repository’s own vocabulary for Hard*, unrelated to (and an unfortunate collision with) KLI’s fork operator κ of §2 — the two never appear in the same formula, but a reader moving between sections should not conflate them:

$$\kappa(0) = \frac{I - \ell}{I}, \quad \kappa(b) = \frac{r_b}{I} \quad (b \in \text{col}_d(y^-)).$$

$\kappa(0)$ agrees with Soft’s identity term, but $\sum_{b \geq 1} \kappa(b)$ need *not* equal ℓ/I — equality holds only at stationarity, $\sum_b r_b = \ell$. Writing α for the population-process rate of the transmission, the auxiliary rates are $\beta_j = \alpha \kappa(j)$, and

$$\Delta\lambda = \alpha \left(1 - \kappa(0) - \sum_{b \geq 1} \kappa(b) \right)$$

is returned to the decay λ ; this may be *negative*, since Hard can propose color changes faster than the population process actually produces them. The realized correction for a drawn outcome j is $\log(\Phi_j/\kappa(j))$ — the α factor cancels, so only the base-law difference from Soft/naive enters the weight. Implemented in `mers_hard.jl/seir_hard.jl` via the `sum_relhaz/relhaz` accessors on a precomputed hazard table (`relhaz_alloc`), with the compensating decay accumulated as `rate - alpha[identity] - alpha[swap-group]`.

5.2 The retracted ε -floor (historical)

This subsection describes a mechanism that is no longer part of the code. It is included, clearly marked, because `mers_filter_suite.tex` §9.1 (which this note otherwise

follows closely) presents it at length, and because understanding *why* it was removed is itself part of the mathematical assessment in §6(v). **Third symbol collision.** The retracted literature’s auxiliary-intensity symbol κ_ε below is, like Hard’s unfloored κ of §5.1, *this repository’s own vocabulary* — it appears in neither KLI nor `pomp` — and collides in the same way with KLI’s fork operator κ (§2); `mers_filter_suite.tex`’s own §9.1 terminology notebbox makes the same point about κ_ε being repository vocabulary.

Prior to v31, all three guided-family kernels mixed their base law π_0 (or $\kappa_{0,\varepsilon}$, for Hard) with a uniform floor over the $\ell + 1$ outcomes,

$$\pi_\varepsilon(j) = (1 - \varepsilon)\pi_0(j) + \frac{\varepsilon}{\ell + 1}, \quad 0 \leq \varepsilon \leq 1,$$

with `proposal_floor` ($= \varepsilon$) defaulting to 0.05. The motivation, as `mers_filter_suite.tex`’s pre-v31 “Full-support proposal” paragraph argued, was a boundary case: at $I = \ell$ (every individual in the source deme already tracked), the naive/Soft identity term $\pi^\emptyset = (I - \ell)/I$ vanishes, and that paragraph asserted that the *target*’s collapsed identity weight $\Phi^\emptyset$ remains positive there, so $B^\emptyset = \Phi^\emptyset/\pi^\emptyset$ would divide by zero — an apparent violation of KLI Theorem 4/5’s requirement that $\pi_u > 0$ wherever $\alpha_u \Phi_u > 0$.

The v31 changelog in `mers_filter_suite.tex` retracts this argument. The reduced compatibility coefficient (KLI Eq. 43) is derived *before* any proposal is chosen, by summing out the inline no-event indicator: the same-deme swap σ_{II}^b and the trivial transition $y' = y$ both produce the same reduced destination, and their contributions collapse to give exactly $(I - \ell)/I$ on both the driver and the target side — so at $I = \ell$, $\Phi^\emptyset$ is *also* zero, not positive as the retracted paragraph claimed. The boundary is therefore a harmless 0/0: `rcateg` never selects a zero-weight outcome, so the boost Φ/π is never evaluated there and no $0 \times \infty$ product ever forms; the corresponding rate mass is charged, exactly, to the decay λ . Replacing π_0 with $\max(\pi_0, \varepsilon/(\ell+1))$ was accordingly never a correction demanded by Eqs. 43–46; it silently defines a *different* importance sampler, one that was never derived or justified as such. (A detailed algebraic derivation of this point is given in the companion note `kli_seirs_pi_note.tex`, referenced by the v31 changelog; this note does not attempt to reproduce that derivation and cites it by name only.) Consequently, as of v31, Soft uses the plain naive split with a guide-weighted *unfloored* tracked-group draw, Guided uses `choose_branch` directly, and Hard uses the raw unfloored auxiliary intensities of §5.1 — and the root/fork proposals use guide-weighted `rcateg` with no feasibility mask: when every weight is genuinely zero, `rcateg` returns the all-zero-weight fallback and the particle receives $-\infty$, which is the correct penalty for a target-incompatible outcome, not a proposal defect.

6 Mathematical assessment in KLI terms

- (i) **Exactness invariance; the ladder is a variance experiment.** Φ_u is a property of the *target* MGP alone — it does not know which of the four kernels is simulating. By KLI Theorem 5/Lemma 3, for *any* π with $\pi_u > 0$ wherever $\alpha_u \Phi_u > 0$, $B_u = \Phi_u/\pi_u$ restores an unbiased estimator of the exact likelihood. Naive, Soft, Guided, and Hard therefore all estimate the same $L(\theta)$; nothing in §5 changes $\hat{L}$ ’s expectation. The entire ladder is a pure variance experiment: which π concentrates its mass closest to where Φ is large.
- (ii) **Guided is the plug-in of the locally optimal proposal.** The one-step optimal proposal at a branch point is $\pi^*(j) \propto \Phi_u(j)g(j)$, where g is the exact backward information filter

— the probability of the *entire remaining, unobserved genealogy and population trajectory* given the post-event coloring j . g is not computable in general (it is exactly the object KLI’s own recursion computes only in aggregate, via Φ , not per-coloring detail conditioned on the future). The finite-state Markov guide of §2 is a *computable mean-field surrogate* for g : it treats each lineage’s deme as evolving independently under a fixed, linear, population-state-*blind* reverse-time process (the guide never reads S, E, I, R or S_c, I_c, S_h, I_h), with no inter-lineage competition for hosts. The relative hazards r_b of §2 are, precisely, the likelihood ratio the surrogate assigns to “recolor b ” versus “leave b alone” at that instant — a plug-in estimate of $g(\text{swap } b)/g(\text{identity})$. Guided’s joint law $\pi(j) \propto w_j, (w_0, r_1, \dots, r_\ell)$, is exactly π^* with this surrogate substituted for g : it is the kernel that takes the guide’s information most literally and uses *all* of it, jointly, in one normalization.

- (iii) **Soft is a KL-suboptimal projection with a variance guarantee.** Soft can be seen as Guided’s law projected onto the constraint set $\{\pi : \pi(\text{tracked group}) = \ell/I\}$ — i.e. Guided, minus its freedom to move mass between the identity and tracked outcomes. Whenever the guide is informative about *which* branch to favor but the aggregate identity/tracked split it implies materially differs from the naive ℓ/I , this projection throws away first-order (KL) efficiency relative to π^* : Soft cannot approach π^* as closely as Guided can, in general. What Soft buys back is a variance *guarantee*: because the tracked-group aggregate is pinned to its naive value regardless of how badly the guide mis-estimates g , the identity/tracked split’s contribution to weight dispersion is capped at exactly the naive kernel’s own level — Soft cannot make that particular source of variance worse than doing nothing, only the within-group allocation can improve on it.
- (iv) **Hard is intensity twisting; it is the only kernel that can fix timing.** Soft and Guided only reallocate proposal mass *at* population-event times that the Gillespie clock already produces at the population rate α ; they cannot change *when* an event fires. Hard can: because its auxiliary intensities $\kappa(j)$ need not sum to the population rate, a strongly favored branch can be made to fire *sooner* than the population process alone would produce it, and less-favored branches later. This is the intensity-twisting idea familiar from Feynman–Kac particle methods (Del Moral [3]) and auxiliary particle filters (Pitt & Shephard [4]): re-weight not just outcomes but the clock itself. The cost is that Hard’s scaling relies on $\sum_b r_b \approx \ell$ (near-stationarity) to keep $\Delta\lambda$ small and well-behaved; this approximation is weakest exactly when $I(t)$ is small relative to ℓ — precisely the regime (few untracked individuals, guide information most decisive) where guidance matters most and where $\Delta\lambda$ can be large and negative, inflating the weight variance Hard was meant to reduce. Hard’s bookkeeping is always exact (the negative $\Delta\lambda$ is a correct accounting of a real intensity shortfall, not an error), but the resulting weights are not thereby lower-variance in this regime.
- (v) **Corrected: absolute continuity holds structurally here, without any floor.** A generic defensive-mixing argument would say: an unfloored proposal risks assigning exact zero probability to a target-feasible outcome, breaking the $\pi_u > 0$ -wherever- $\alpha_u \Phi_u > 0$ hypothesis of KLI Theorem 5 and biasing $\hat{L}$ downward. **This argument does not apply to the guide architecture of §2/5.1.** Trace where an exact zero can actually arise in this code:
 - *Data pins.* **demekron!** writes an exact Kronecker delta at a genealogy node where **knowledge!** certifies the deme (a MERS sample’s host, an SEIRS sample or internal node’s Infec membership). These are zero-elapsed-time certificates, not approximations: they assert that this specific lineage *is*, with probability one, in this deme at this instant,

because the data say so. If this pin later makes some branch-choice weight exactly zero, that outcome is already excluded by the data itself — a particle that reached it would die at the very next data-consistency check regardless of how it got there. Flooring such a weight would not rescue a salvageable particle; it would spend proposal mass reviving one that the data have already condemned.

- *Guide-chain reducibility.* If the user-supplied `fsmarkov` conductances make the guide process reducible (e.g. a zero Camel $\leftrightarrow$ Human conductance, mirroring $\beta_{ch} = 0$ in the population model), some transition’s relative hazard is identically zero on structural grounds. Here too the zero is a certificate about the *guide’s own* model, not noise: an informative guide should be built to reflect which cross-deme moves are reachable, and a genuinely irreducible target with a reducible guide is a guide *misspecification* to fix by choosing better conductances, not a case for defensive mixing at the proposal stage.
- *Everywhere else: strictly positive by construction.* Away from a pin and away from guide-chain reducibility, `forward_action` applies $P(s) = L \text{diag}(e^{\Lambda s}) R$ to a nonnegative belief vector for $s > 0$; for an irreducible generator this transition semigroup is strictly positive for every $s > 0$ (the standard positivity property of an irreducible continuous-time chain’s transition matrix), so every belief propagated across a positive-length interval, and every relative hazard r_b derived from it, is strictly positive. Absolute continuity therefore holds *structurally*, node by node, without any floor to enforce it.

The only residual caveat is numerical, not statistical: on an extremely long, strongly pinned interval, $e^{\Lambda s}$ can underflow in double precision before it reaches a genuine zero, producing a proposal weight indistinguishable from zero in floating point. This is a log-space/numerical-stability concern to monitor (e.g. working in log-hazards on long branches), not a mixing concern to paper over with ε -floor mass. The SEIRS kernels have never had a floor and are not observed to need one; the MERS kernels’ v31 removal (§5.2) brings MERS into the same, already-validated pattern.

(vi) **Two structural approximation limits of the guide itself.** Two places where the guide’s mean-field construction departs from the true backward filter g , independent of which of the four kernels is used to consult it:

- (a) *Conditional independence at a fork.* The fork proposal’s product form `branch_options`(x) = $[x_{1,1}x_{2,2}, x_{1,2}x_{2,1}]$ (SEIRS) and the analogous g_1, g_2, g_3 products (MERS, §4) assume the two children’s subtree beliefs are conditionally independent given the parent’s deme. This is *exact* for the linear guide process itself (each lineage genuinely evolves independently under `FSMarkovProc`), but only *approximate* for the true target conditional law, in which the two subtrees can share information through the shared, density-dependent population trajectory (e.g. a large I_c implied by one subtree’s sampling pattern is informative about the other subtree’s plausible colorings too, a coupling the guide cannot see).
- (b) *State-homogeneous guide vs. state-dependent target.* The guide is genealogy-*inhomogeneous* (it depends on tree topology and node times through the backward sweep) but population-*state-homogeneous*: its transition rates are fixed constants, chosen once via `fsmarkov`(...), and never see the live $I_c(t), I_h(t)$. For MERS, the decision the true backward filter would weight most heavily — how much more likely a lineage’s ancestor was Camel versus Human — properly depends on the *current* reservoir sizes $I_c(t)$ and $I_h(t)$, which are strongly

asymmetric and time-varying in the epidemic (e.g. a large, long-lived camel reservoir versus small, short spillover clusters in humans). The guide’s fixed conductances capture this asymmetry only *on average*, over the whole trajectory, not at the specific t where a decision is being made.

Both (a) and (b) are exactly the residual $\text{KL}(\pi^* \parallel \pi_{\text{Guided}})$ left over after the mean-field plug-in of point (ii): the guide reproduces π^* ’s *first-moment* structure (which branch is favored, on average) but not its state-dependent covariance structure. This residual governs the estimator’s variance only at *second* order: the weight variance under a proposal π relative to the Φ -twisted target is controlled by the χ^2 -divergence $\chi^2(\Phi \cdot \parallel \pi) = \mathbb{E}_\pi[(\Phi/\pi)^2] - 1$ (a standard importance-sampling identity), of which the KL divergence used above is the natural *first-order* surrogate near $\pi \approx \Phi$ -proportional; both diagnose the same mean-field gap, but χ^2 , not KL, is the quantity that actually sets $\text{Var}(\hat{L})$ to leading order.

7 Verdict on the MERS kernel choice

We agree that the repository’s current default — GuidedMERS, unfloored, matching the SEIRS pattern — is the right choice, and that the v31 removal of the ε -floor was mathematically correct.

Why Guided. Point (ii) makes this the principled choice among the three guided-family kernels: Guided is the direct plug-in of the locally optimal proposal π^* using the guide as a surrogate for the intractable backward filter g , using *all* of the guide’s information jointly rather than only its within-group ranking (Soft) or its raw, unnormalized intensities (Hard). Whenever the guide is informative, Guided dominates Soft in KL terms by construction (point iii); Hard’s intensity-twisting freedom (point iv) is a genuine additional capability but one whose benefit is undermined exactly where it would matter most (small $I(t)$, near-stationarity assumption weakest), so it does not obviously dominate Guided on this model.

Why unfloored. Point (v) is not a hedge: the removal is not merely tolerable, it is the mathematically justified state, given how this guide is built. Every exact zero in a `seir_guided.jl/mers_guided.jl` proposal traces to a data pin or a guide-chain reducibility, both of which are certificates that the corresponding outcome is already target-incompatible or already doomed, not artifacts of an approximate surrogate running short of support. Paying variance for a floor that resurrects only already-doomed particles has no compensating benefit.

Kernel choice is not, in fact, the binding constraint on the real MERS tree. The empirical diagnostic recorded in `mers_kernel_diagnostic_findings.md` confirms this directly, and independently of the theory above: on the full 274-tip Dudas et al. [8] genealogy, *all four kernels collapse identically* — Naive, Soft, Guided, and Hard each give exactly 100 of 548 filtering steps equal to $-\infty$, the first at step 144, for every seed and every parameter variation tested. Every one of the 100 failures is a Camel-deme sample; direct state inspection confirms the simulated camel-infected count I_c has already gone stochastically extinct beforehand, and stays there permanently because $\beta_{ch} = 0$ and $B_c = 0$ leave no way to regenerate a camel infection once $I_c = 0$ — a true absorbing state under Table 2 parameters. A second, simpler collapse mode occurs under the module’s bare defaults ($\chi_h = 0$): the very first Human sample node forces $\log(\chi_h I_h) = \log 0 = -\infty$ unconditionally, before extinction or any other dynamics get a chance to matter. Both are $\Phi = 0$,

target-level incompatibilities — exactly the boundary KLI Theorem 5 draws around what *any* admissible proposal kernel may and may not repair. No amount of guide sophistication, and no re-litigation of Soft vs. Guided vs. Hard, changes a target-level $\Phi = 0$; paying Hard’s variance premium in particular buys nothing on this tree, since the binding constraint is upstream of the proposal entirely.

Honest caveats and a concrete next step. None of the above should be read as “the current setup already filters the real tree.” It does not, for the reason just given, and fixing that is a prerequisite for any kernel comparison to be meaningful on real data:

1. **Fix the parameter-support issues first.** A nonzero χ_h (so the first human sample is not an automatic $-\infty$) and a regeneration path for camel infection — $\beta_{ch} > 0$ or $B_c > 0$, either of which breaks the $I_c = 0$ absorbing state — are prerequisites for *any* kernel to filter the real tree at all. This is parameter/model-regime work, not proposal-design work (`mers_kernel_diagnostic_findings.md` reaches the same conclusion and further suggests a non-destructive-sampling or waning-immunity camel variant, e.g. Yang’s SIRS formulation, as a structural alternative to an endemic reservoir that goes extinct on a multi-year tree).
2. **If ESS is still poor near camel-sample clusters at low I_c once support is fixed, the next rung is not Hard-as-implemented.** Point (vi)(b) identifies the dominant residual: the guide’s fixed conductances average over the whole trajectory instead of reflecting the *current* $I_c(t), I_h(t)$. The natural fix attacks that gap directly, rather than reaching for Hard’s timing freedom: a **state-dependent guide function** correcting the FSMarkov guide’s stationarity normalization $\sum_b r_b = \ell$ by the current deme occupancies $I_d(t)$, in the sense of the guided intermediate resampling filter (GIRF; Park & Ionides [5]) — a GIRF-type twist localized to the next sample event’s known host, rather than a generic lookahead. (“State-adaptive guided kernel” is only this note’s shorthand for the same idea; GIRF’s guide function is the established term and is preferred.) This narrows the mean-field gap of point (vi)(b) directly, rather than trading it for Hard’s timing-freedom variance premium of point (iv); the general idea of twisting a filter by a lookahead correction is shared with the auxiliary particle filter (Pitt & Shephard [4]) and, in a fully iterated form, with the iterated auxiliary particle filter (Guarniero, Johansen & Lee [6]) and controlled SMC (Heng et al. [7]).
3. **Keep Hard as a diagnostic instrument, not a production kernel.** Hard’s weight variance localizes precisely where the guide’s timing information is poor (point iv); that makes it a useful probe for finding such regions during development, but not a kernel to prefer for production likelihood estimates on this model.

8 Summary table

Addendum (18 September 2026)

Four developments since the analysis above, none of which changes the verdict of §7:

1. **mers_guided.jl rewritten upstream, now self-contained.** Aaron King rewrote `mers_guided.jl` (module `GuidedMERS`) so that it no longer includes `mers_funs.jl`; its architecture now mirrors `seir_guided.jl`’s directly, with its own `knowledge!`, `singular_part!`, and modular rate pieces (`transmission!`, `removal!`, `demography!`, `sampling`) composed by `event_rates!`.

Kernel	Base law	What the guide controls	Weight-variance mechanism / when preferred
Naive	$\pi(0) = (I - \ell)/I$, $\pi(b) = 1/I$ uniform	Nothing — guide not consulted	Baseline; correct but leaves all guide information on the table.
Soft	$\pi(0) = (I - \ell)/I$ fixed; within-group $\pi(b) \propto r_b$	Which tracked branch, not the identity/tracked split	Caps the identity/tracked split’s variance contribution at the naive level; KL-suboptimal relative to π^* whenever the guide implies a different split.
Guided	Joint $\pi(j) \propto (I - \ell, r_1, \dots, r_\ell)$	Identity vs. tracked <i>and</i> which branch, jointly	Plug-in of the locally optimal $\pi^* = \Phi \cdot g$ with the guide as surrogate for g ; dominates Soft in KL whenever the guide is informative. <i>Repository’s MERS default.</i>
Hard	$\kappa(0) = (I - \ell)/I$, $\kappa(b) = r_b/I$, unnormalized; $\Delta\lambda$ to decay	Event <i>timing</i> as well as outcome, via auxiliary intensity twisting	Only kernel that can move event times, not just reallocate at fixed times; benefit degrades where $\sum_b r_b \not\approx \ell$ (small $I(t)$), which is exactly where guidance matters most. Useful diagnostically.

Table 1: The four-kernel ladder. All four target the same Φ and are unbiased for $L(\theta)$ by KLI Theorem 5/Lemma 3; only the proposal-induced variance differs. *Retracted (v31)*: all three guided-family kernels previously mixed their base law with a uniform ε -floor, $\varepsilon = 0.05$ by default (§5.2); this mechanism has been deleted from every MERS kernel and is not reflected in this table.

The proposal remains “semisoft”: at a fork the orientation and within/cross-host type are drawn jointly from guide-weighted rates, and regular-time cross-host events use `choose_branch` exactly as described in §4. References to `mers_funs.jl` in §2 and §4 above have been updated accordingly.

2. `mers_funs.jl` now shared only by SoftMERS/HardMERS. These two are being refactored toward GuidedMERS’s nested-state (`state=(;S_c,I_c,S_h,I_h)`, `live::Bool`) architecture, differing from it only in `regular_part!/event_rates!`.
3. **An upstream fork-term bug, since patched in this repository’s copy.** Aaron’s rewritten `singular_branch!` charged $-\log(I_C(I_C - 1))$ (resp. I_H) at a same-deme fork; the correct factor, matching `mers_naive.jl`, the pre-rewrite `mers_funs.jl`, and the compiler IR’s `ForkTransition` $\varphi = 2/(I(I - 1))$, is $-\log(I_C(I_C - 1)/2)$ — exactly the factor already used in §4 above and derived throughout `mers_filter_suite.tex`. The missing $/2$ contributes a constant $\log 2$ per within-host coalescence; on an all-camel 4-tip tree (3 internal nodes) the unpatched and patched log-likelihood estimates differed by 2.04 ± 0.43 , against the predicted $3 \log 2 = 2.08$. When the data force the host assignment at every internal node (as on that all-camel tree) this is a pure offset; on a mixed tree the likelihood sums over colorings and

the missing $/2$ halves the within-host component ($\propto \beta_{CC}, \beta_{HH}$) while leaving the cross-host components ($\propto \beta_{HC}, \beta_{CH}$) intact, tilting the surface toward cross-host explanations. It is to be relayed to Aaron as an upstream fix.

4. **Dependency updates.** The package now depends on registered `PartiallyObservedMarkovProcesses` v0.10 (previously a 0.8.1-dev pin to the `devel` branch); `Distributions` is now a direct dependency; `mif`, `geometric_cooling`, and `traces` are new POMP exports exercised by `test/mers_guided.jl`. `guide.jl` dropped its `@inbounds` annotations to match upstream, and `mers_tree.jl` no longer defines `mers_trees` (only `mers_newick`).
5. **A second upstream bug, since patched in this repository’s copy.** Aaron’s `regular_transmission_cc` (regular-time within-host births) returned a zero log-weight, omitting the “no visible fork” factor $1 - \binom{\ell_d}{2} / \binom{I_d}{2}$ that `mers_naive.jl`, `Soft/Hard`, and the compiler’s Chu–Vandermonde aggregate ($\Phi_{\text{identity}} + \ell \Phi_{\text{inline}} = 1 - \binom{\ell}{2} \Phi_{\text{fork}}$) all charge on a within-deme birth that does not coalesce two tracked lineages. This one is *state-dependent* even when every node’s host assignment is forced by the data — it scales with the number of within-host births while $\ell_d \geq 2$ — so it biases parameter inference strongly (the log 2 fork-term bug, item 3 above, is a pure offset only in that forced case), not merely the absolute likelihood. Verified on a 4-camel-tip tree with long coexisting branches ($\beta_{cc} = 3$, $N_c = 40$, $I_{c0} = 0.25$; 30 reps of $N_p = 4000$ each): the pre-rewrite guided kernel gave -28.63 ± 0.04 , Aaron’s version with only the log 2 fix gave -22.58 ± 0.30 , and Aaron’s version with both fixes restored gave -28.58 ± 0.05 . This repository’s copy now carries both fixes; both are to be relayed to Aaron as upstream fixes.

References

- [1] King, A. A., Lin, Q., and Ionides, E. L. *Exact phylodynamic likelihood via structured Markov genealogy processes*. arXiv:2405.17032 (posted 2024, revised 2026).
- [2] King, A. A., Nguyen, D., and Ionides, E. L. Statistical inference for partially observed Markov processes via the R package `pomp`. *Journal of Statistical Software* 69(12):1–43, 2016.
- [3] Del Moral, P. *Feynman–Kac Formulae: Genealogical and Interacting Particle Systems with Applications*. Springer, 2004.
- [4] Pitt, M. K., and Shephard, N. Filtering via simulation: auxiliary particle filters. *Journal of the American Statistical Association* 94(446):590–599, 1999.
- [5] Park, J., and Ionides, E. L. Inference on high-dimensional implicit dynamic models using a guided intermediate resampling filter. *Statistics and Computing* 30:1497–1522, 2020.
- [6] Guarniero, P., Johansen, A. M., and Lee, A. The iterated auxiliary particle filter. *Journal of the American Statistical Association* 112(520):1636–1647, 2017.
- [7] Heng, J., Bishop, A. N., Deligiannidis, G., and Doucet, A. Controlled sequential Monte Carlo. *Annals of Statistics* 48(5):2904–2929, 2020.
- [8] Dudas, G., Carvalho, L. M., Rambaut, A., and Bedford, T. MERS-CoV spillover at the camel-human interface. *eLife* 7:e31257, 2018.